```
    def "should calculate the correct arrival time"() {
        given:
            itineraryService = new ItineraryService();
        when:
            def proposedTrainTimes =
                   itineraryService.findNextDepartures("Parramatta",
                                                       "Town Hall",
                                                       at('8:00'));
        then:
            proposedTrainTimes == [at('8:02'), at('8:11'), at('8:14' )]
    }

    def at(String time) {
        def hour = Integer.valueOf(time.split(':')[0])
        def minute = Integer.valueOf(time.split(':')[1])
        new LocalTime(hour.toInteger(), minute.toInteger())
    }
}
```

The requirement you're checking

❶ **"Given I have an itinerary service"**

❷ **"When I find the next departures"**

❸ **"Then I should get these proposed times"**

A utility function to make the time values easier to initialize

This specification does essentially the same thing as the JBehave test. It creates a new itinerary service ❶, finds the next departures between Parramatta and Town Hall starting from 8:00 am ❷, and checks that it matches the times you expect ❸. When this passes, you can be confident that the `ItineraryService` class does what you expect it to do.

Of course, if you run this specification, it will fail, because you haven't written any application code yet. It's now time to write that code. Let's see what the `findNextDepartures()` method might look like:

```
public List<LocalTime> findNextDepartures(String departure,
                                          String destination,
                                          LocalTime startTime) {
  List<Line> lines = timetableService.findLinesThrough(departure,
                                                       destination);
  List<LocalTime> allArrivalTimes = getArrivalTimesOnLines(lines,
                                                           departure);
  List<LocalTime> candidateArrivalTimes
                    = findArrivalTimesAfter(startTime, allArrivalTimes);
  return atMost(3, candidateArrivalTimes);
}

private List<LocalTime> atMost(int max, List<LocalTime> times) {
        return Lists.partition(times, max).get(0);
}
```

❶ **Which lines go through these two stations?**

❷ **What time do trains on these lines get to the departure station?**

❸ **What trains arrive here after the start time?**

❹ **I only want to know about the first three of them.**

A convenience method to return up to three arrival times.

The job of the itinerary service is to calculate information about departure times and trip details based on the current timetables, and to do this, it will need timetable data. But you haven't written the code to figure out this bit yet: timetables are complicated things and represent a separate area of concern. Following good design practices, you can use a separate service to provide timetable details to the itinerary service. The preceding code introduces a `timetableService` object ❶ to take care of this.